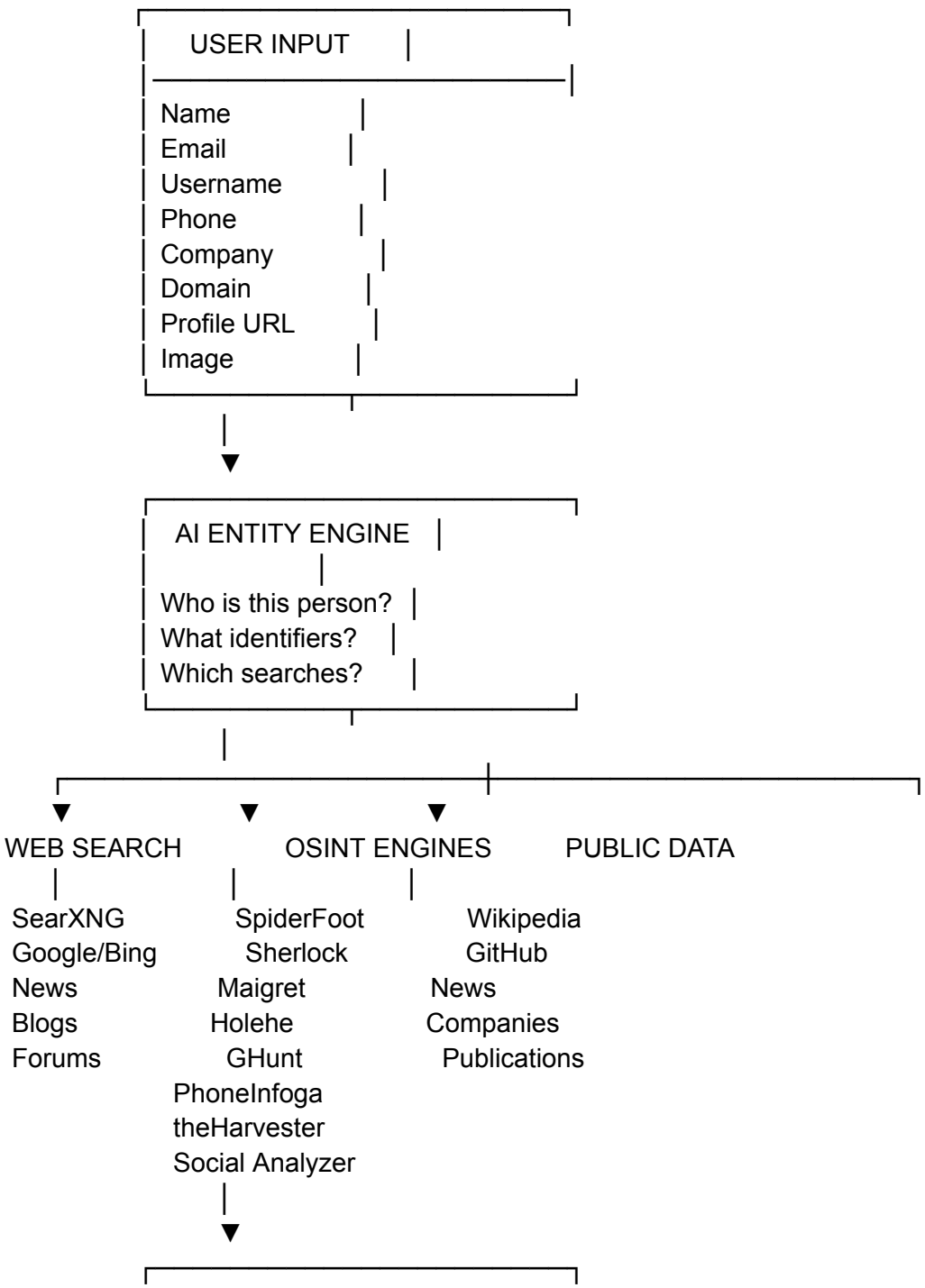
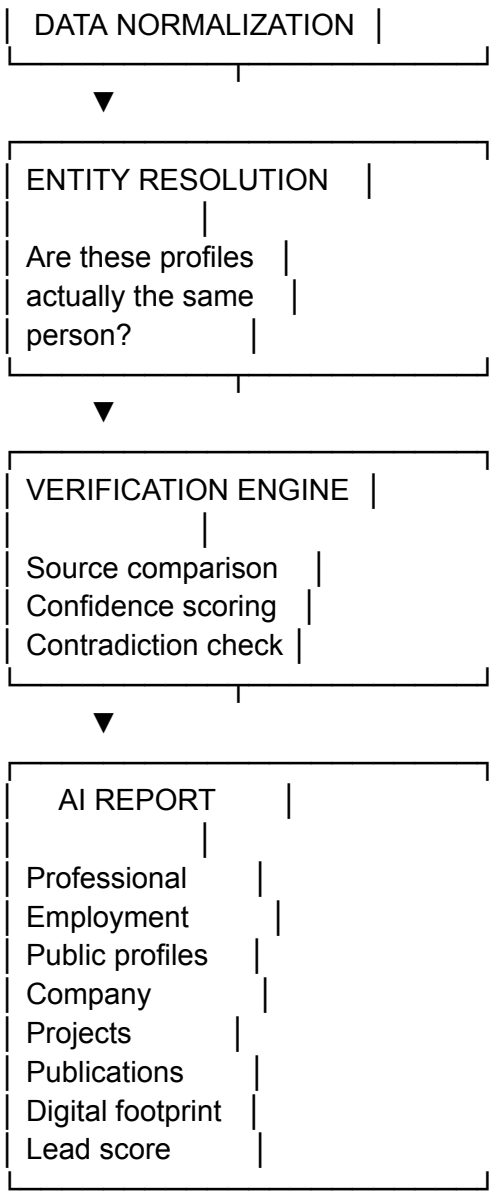
The system I'd build

Call it something like:

UNIVERSAL INTELLIGENCE ENGINE





Why SearXNG is useful

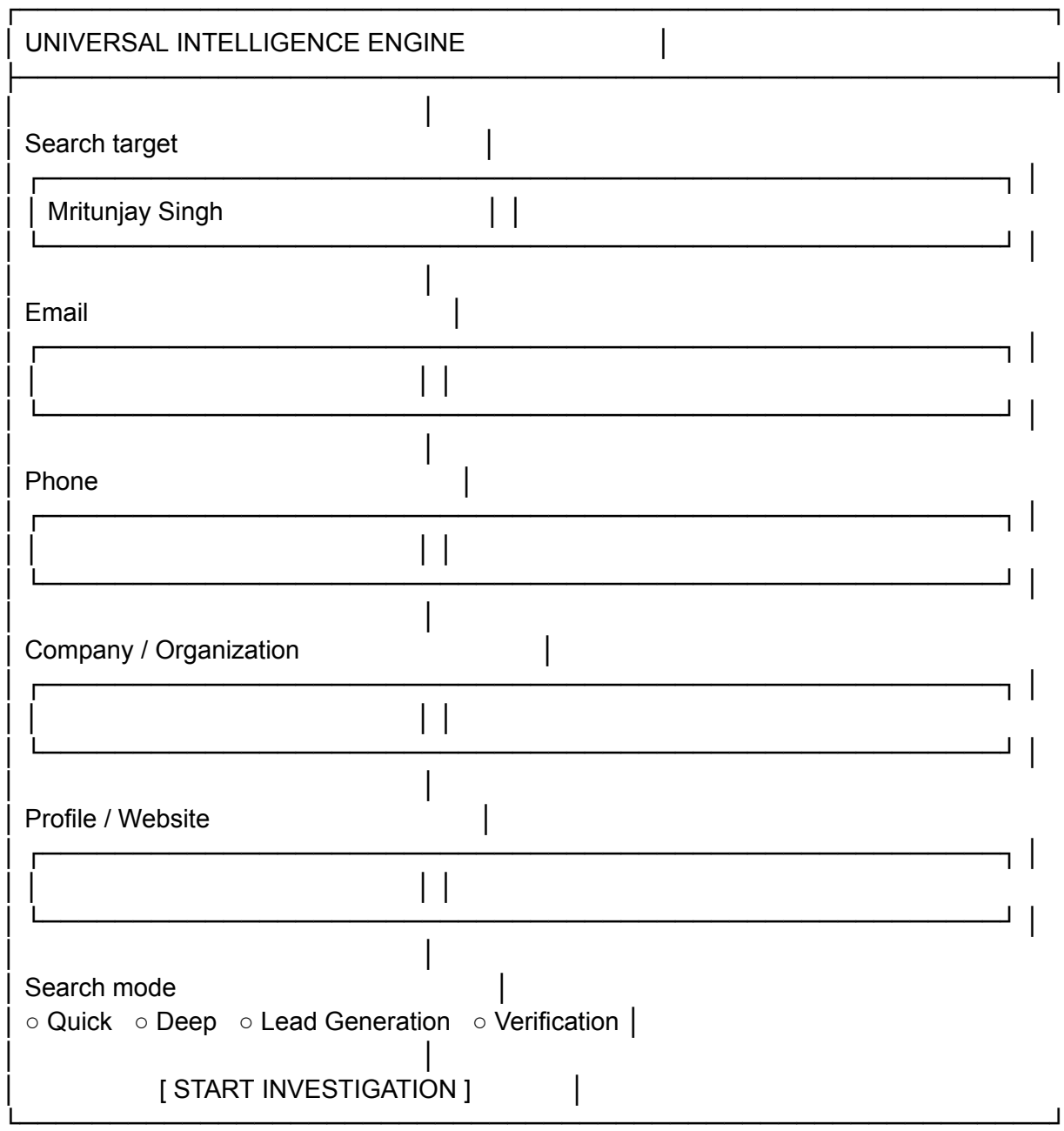
I'd make a **self-hosted SearXNG instance** your web-search layer. It has an HTTP search API and can return JSON/CSV/RSS, making it straightforward for your backend to query programmatically.

Then your OSINT tools become **workers**, rather than separate applications the user has to operate.

Your dashboard

The front end should have a single investigation form.

Input



Then show a live progress screen:

INVESTIGATION RUNNING

✓ Input normalization
✓ Identity candidates
✓ Web search
✓ GitHub search
✓ News search
✓ Company search
✓ Username discovery
✓ Email checks
✓ Public-profile discovery
✓ Entity correlation
● AI verification
○ Report generation

The killer feature: AI orchestration

Don't hard-code:

> "Every search always runs every tool."

Instead have an AI **Research Planner**.

For example:

INPUT:

Name = John Smith
Company = Acme AI

AI decides:

Relevant modules:

✓ General web
✓ News
✓ Company
✓ GitHub
✓ Username discovery
✓ Professional sources
✓ Public publications

Skip:

✗ Phone investigation
✗ Email investigation

Reason:
No phone/email supplied.

If the user provides:

john@example.com

it activates email-related modules.

If they provide:

+91XXXXXXXXXX

it activates phone-related **public-information** modules.

That makes your system much faster.

Your tool registry

This is where the architecture becomes powerful.

Create:

```
/tools
   /web_search
   /github
   /wikipedia
   /news
   /company
   /username
   /email
   /phone
   /domain
   /image
```

Every tool has the same interface:

```
{
  "tool": "github_search",
  "input": {
```

```
    "name": "John Smith",
    "company": "Acme AI"
  }
}
```


and returns:


```
{
  "source": "GitHub",
  "results": [],
  "confidence": 0.82,
  "timestamp": "...",
  "evidence": []
}
```


Now you can add 100 tools without changing the dashboard.

Core OSINT layer

I'd start with:

SpiderFoot

This is probably the closest existing open-source foundation to what you're imagining. It already provides an OSINT automation platform with a web interface, modules, database, API, scheduler, correlation and reporting capabilities.

Sherlock

Username discovery.

Maigret

Broader username/profile discovery.

Holehe

Email-to-service presence checks where supported.

GHunt

Public Google-account-related OSINT.

PhoneInfoga

Phone-number reconnaissance.

theHarvester

Domain/company reconnaissance.

Photon

Website crawling.

Social Analyzer

Public social-profile discovery.

Recon-ng

Modular reconnaissance framework.

Instead of making each one visible to your user, your dashboard becomes the **control plane**.

The AI layer

I'd give the AI four separate jobs.

1. Planner

Determines what searches to perform.

2. Correlator

Determines whether:

John Smith
johnsmith42
john@acme.com
github.com/johnsmith42
linkedin.com/in/john-smith

are likely connected.

3. Fact checker

Never blindly trust one result.

For example:

Source A → Software Engineer
Source B → Senior Software Engineer
Source C → CTO

The AI should say:

> Conflicting job titles detected.

rather than inventing a definitive answer.

4. Report generator

Produces:

IDENTITY
Professional Profile
Employment
Education
Companies
Public Projects
GitHub
Public Social Profiles
Publications
News
Web Presence
Company Intelligence
Lead Qualification
Evidence
Confidence

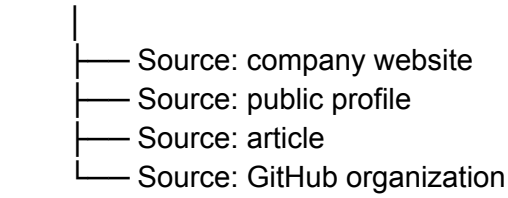
Very important: evidence-first architecture

This is what I'd add that many OSINT tools lack.

Every claim gets:

claim
source
URL
date discovered
source type
confidence
supporting evidence

For example:

Current employer


Confidence: 93%

That makes the system useful for **verification**, rather than merely generating an AI biography.

Lead-generation mode

This is where you can make your custom system better suited to Apollo.

Instead of searching one person:

Find CTOs at AI startups in India

Your agent can:

Discover companies
↓
Find public professional contacts
↓
Identify relevant decision makers
↓
Enrich permitted business information
↓
Verify
↓
Score

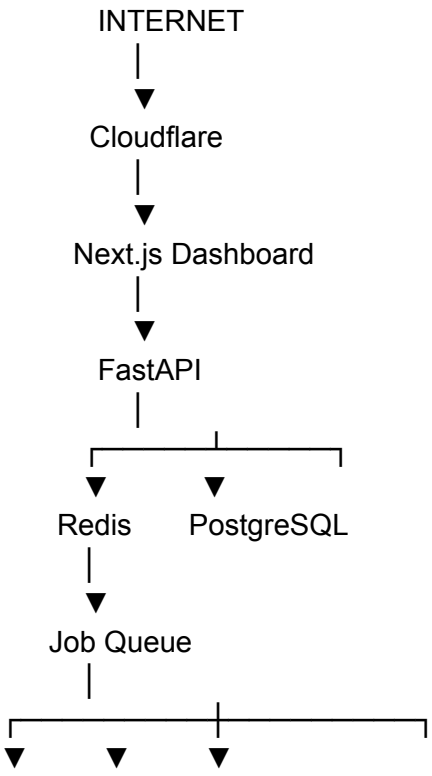
↓
Generate personalized research
↓
Export CSV/CRM

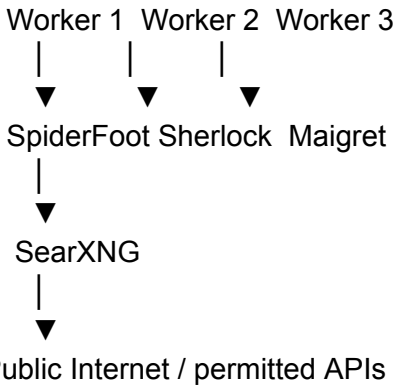
Your lead score could be:

ICP Match	25
Decision-maker	20
Company size	15
Industry relevance	15
Technology match	10
Recent activity	5
Public business email	5
Evidence confidence	5
TOTAL	100

Cloud architecture

If you don't want to run it on your laptop, I'd deploy it like this:





Cloud choices

For a first version:

Frontend: Vercel
Backend: Railway / Render / Fly.io
Database: Supabase PostgreSQL
Redis: Upstash
Workers: Docker containers
SearXNG: VPS
Object storage: Cloudflare R2
Monitoring: Sentry
AI: OpenAI API or local model through Ollama

You could also put practically everything on a single VPS initially and split services later.